Lecture-2 (Purpose of a Database System)

Purpose of a Database System

A **database system** is used to store data safely, retrieve it easily, and manage it efficiently.

Main Purposes

1. Store data permanently

Data stays safe even after power is off.

2. Organize data properly

Tables, rows, and columns keep data neat and structured.

3. Easy and fast data retrieval

You can quickly search or filter information.

4. Avoid data duplication

Database removes repeated data using normalization.

5. Ensure data accuracy (integrity)

Data rules keep information correct and consistent.

6. Provide security

Only authorized users can access or change data.

7. Support multiple users

Many users can use the system at the same time without problems.

8. Backup and recovery

Data can be restored if something goes wrong.

What is a Data Model?

A **data model** is a **way to organize and represent data** in a database.

It shows how data is stored, connected, and used.

Simple Example

- **Student Database:**
  - Students → have ID, Name, Age
  - Courses → have CourseID, Name
  - Students **enroll in** Courses

The **data model** shows **tables and relationships** between students and courses.

Let's take a **Student Table** as an example:

This table shows **how data is organized** in a database.

- **StudentID** → Primary Key (PK), unique for each student
- **Name, Age, Course** → Attributes of the Student relation

Relational Model Description

1. **Relation (Table):** Student
2. **Attributes (Columns):** StudentID, Name, Age, Course
3. **Tuple (Row):** Each student record
4. **Primary Key:** StudentID ensures uniqueness

This table is a **simple relational model** showing how data is stored and uniquely identified.

Levels of Abstraction in a Database

Database abstraction **hides unnecessary details** and shows only what the user needs. There are **three levels**:

---

1. External Level (User View)

- What **users see**
- Each user can have a **different view**
- Hides irrelevant details

Example:

A student sees only **his marks**, not other students' data.

---

2. Conceptual Level (Logical View)

- Shows the **whole database structure** logically
- Includes **tables, relationships, constraints**
- Independent of physical storage

Example:

Tables: Students, Courses, Marks

Shows which students take which courses.

---

3. Internal Level (Physical View)

- How data is **actually stored** on disk
- Includes files, indexes, storage blocks
- Deals with **performance and efficiency**

Example:

Data stored in **binary files**, indexed by StudentID.

Summary Table

Level	What It Shows	Example
External	User-specific view	Student sees only his marks
Conceptual	Full logical database view	Tables & relationships
Internal	Physical storage details	Files, indexes, storage blocks

Instances and Schemas

1. Schema

- **Definition:** The **structure or design** of the database
- **Static:** Doesn't change frequently
- **Includes:** Tables, columns, data types, relationships, constraints

Example:

For a Student database:

- Table: `Student(StudentID, Name, Age, Course)`
- Table: `Course(CourseID, Name, Teacher)`

This **design** is the schema.

---

2. Instance

- **Definition:** The **actual data stored** in the database at a particular moment
- **Dynamic:** Changes when data is inserted, updated, or deleted

Example:

StudentID	Name	Age	Course
1	Ali	20	CSE
2	Sara	21	EEE

This **current data** is an instance of the schema.

Key Difference Table

Feature	Schema	Instance
Meaning	Structure/design of database	Actual data at a moment
Changes	Rarely changes	Changes frequently
Type	Static	Dynamic
Example	Table structure, columns	Records like Ali, Sara

Physical Data Independence

Definition:

Physical data independence is the **ability to change the internal (physical) storage of data** without affecting the **conceptual schema** or the users' view of the database.

Explanation in Simple Words

- You can **change how data is stored** (like using different file structures, indexing, or storage devices)
- **User applications and queries don't need to change**
- Helps **maintain flexibility and efficiency**

Example

- Original storage: Student table stored in **heap file**
- Change storage: Move Student table to **B-tree indexed file**

- Users querying the student table (**SELECT Name FROM Student**) **don’t notice any change**

Here, the **conceptual schema remains the same**, but the **physical storage changed**. That’s physical data independence.